8-Puzzle A* using Manhattan Distance

Code:

```
import heapq

GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)

class Node:
    def __init__(self, state, g, h, parent=None):
        self.state = state
        self.g = g  # Depth
        self.h = h  # Manhattan distance sum
        self.f = g + h
        self.parent = parent

    def __lt__(self, other):
        return self.f < other.f

def get_neighbors(state):
    neighbors = []
    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        nr, nc = row + dr, col + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_idx = nr * 3 + nc
            state_list = list(state)
            state_list[zero_idx], state_list[new_idx] = state_list[new_idx],
state_list[zero_idx]
            neighbors.append(tuple(state_list))
    return neighbors

def calculate_manhattan(state, goal):
    # h(n): Sum of horizontal and vertical distances
    distance = 0
    for i in range(9):
        val = state[i]
        if val != 0:
            curr_r, curr_c = divmod(i, 3)
            goal_idx = goal.index(val)
            goal_r, goal_c = divmod(goal_idx, 3)
            distance += abs(curr_r - goal_r) + abs(curr_c - goal_c)
    return distance

def solve_manhattan(initial_state):
    open_set = []
    initial_node = Node(initial_state, g=0, h=calculate_manhattan(initial_state,
GOAL_STATE))
    heapq.heappush(open_set, initial_node)
    closed_set = set()
```

```
    while open_set:
        current = heapq.heappop(open_set)

        if current.state == GOAL_STATE:
            path = []
            while current:
                path.append(current)  # Append node instead of state
                current = current.parent
            return path[::-1]

        closed_set.add(current.state)

        for neighbor in get_neighbors(current.state):
            if neighbor in closed_set:
                continue
            g = current.g + 1
            h = calculate_manhattan(neighbor, GOAL_STATE)
            heapq.heappush(open_set, Node(neighbor, g, h, parent=current))

    return None

def print_solution_steps(path):
    if not path:
        print("No solution found.")
        return

    print(f"Manhattan Distance - Steps: {len(path) - 1}\n")
    for step, node in enumerate(path):
        header = "Step 0 (Initial State)" if step == 0 else f"Step {step}"
        print(f"{header} | f(n) = {node.f} (g(n) = {node.g}, h(n) = {node.h})")

        # Format tuple into 3x3 grid display
        for i in range(0, 9, 3):
            print(" ".join(str(x) for x in node.state[i:i+3]))
        print()

# Test Run
if __name__ == "__main__":
    initial = (1, 2, 3, 4, 0, 5, 6, 7, 8)
    path = solve_manhattan(initial)
    print_solution_steps(path)
```

Output:

```
PS C:\Users\Administrator\Desktop\1BM24CS316> & c:\python314\python.exe c:/Users/Administrator/Desktop/1BM24CS316/AStarManhattan.py
Manhattan Distance - Steps: 14

Step 0 (Initial State) | f(n) = 6 (g(n) = 0, h(n) = 6)
1 2 3
4 0 5
6 7 8

Step 1 | f(n) = 6 (g(n) = 1, h(n) = 5)
1 2 3
4 5 0
6 7 8

Step 2 | f(n) = 8 (g(n) = 2, h(n) = 6)
1 2 3
4 5 8
6 7 0

Step 3 | f(n) = 10 (g(n) = 3, h(n) = 7)
1 2 3
4 5 8
6 0 7

Step 4 | f(n) = 10 (g(n) = 4, h(n) = 6)
1 2 3
4 5 8
0 6 7

Step 5 | f(n) = 12 (g(n) = 5, h(n) = 7)
1 2 3
0 5 8
4 6 7

Step 6 | f(n) = 14 (g(n) = 6, h(n) = 8)
1 2 3
5 0 8
4 6 7

Step 7 | f(n) = 14 (g(n) = 7, h(n) = 7)
1 2 3
5 6 8
4 0 7

Step 8 | f(n) = 14 (g(n) = 8, h(n) = 6)
1 2 3
5 6 8
4 7 0

Step 9 | f(n) = 14 (g(n) = 9, h(n) = 5)
1 2 3
5 6 0
4 7 8
```

```
Step 10 | f(n) = 14 (g(n) = 10, h(n) = 4)
1 2 3
5 0 6
4 7 8

Step 11 | f(n) = 14 (g(n) = 11, h(n) = 3)
1 2 3
0 5 6
4 7 8

Step 12 | f(n) = 14 (g(n) = 12, h(n) = 2)
1 2 3
4 5 6
0 7 8

Step 13 | f(n) = 14 (g(n) = 13, h(n) = 1)
1 2 3
4 5 6
7 0 8

Step 14 | f(n) = 14 (g(n) = 14, h(n) = 0)
1 2 3
4 5 6
7 8 0

PS C:\Users\Administrator\Desktop\1BM24CS316>
```